

VIBE CODING ・ 実践教科書

バイブコーディング 自己修正の教科書

AIに任せて作る時代に、
「自分でエラーを直せる人」になるための最短ルート。

- ◆ AIが壊したコードを、自力で立て直せるようになる
- ◆ エラーメッセージが「怖いもの」から「地図」に変わる
- ◆ 推測で時間を溶かすデバッグから卒業する

最短 ・ 最大効率で学ぶ

初学者～独学者のための実践ガイド

全11章 + 付録

テンプレート ・ チェックリスト付き

目次

1 序章 —— なぜ「自己修正力」が9割なのか

この本が解く問題と、最短で身につく地図

2 マインドセット —— あなたは「監督」である

AIは有能な部下。指示し、確認し、責任を持つのはあなた

3 安全網を最初に張る —— Gitと小さなコミット

「壊しても戻せる」が、自由に試せる勇気になる

4 エラーメッセージを読む技術

9割のエラーは、メッセージ自身が答えを書いている

5 体系的デバッグ —— 推測をやめ、検証する

二分探索 ・ 最小再現 ・ 仮説検証ループ

6 AIに正しく指示する —— コンテキスト設計

出力の質は、入力で決まる

7 コードを「読める」最低限

全部書けなくていい。何が起きているか追えればいい

8 検証とテスト —— 「動いた」を証明する

目視確認から、自動テストの第一歩まで

9 よくある罣カタログ

ハルシネーション ・ 依存地獄 ・ セキュリティ穴

10 実践ワークフロー —— 1機能を完成させる流れ

小さく刻み、一周ずつ確実に進める

11 テンプレート & チェックリスト集

今日からコピペで使える実戦装備

12 付録 —— 用語集 / 30分クイックスタート

つまずきやすい言葉を、ひと言で

序

なぜ「自己修正力」が9割なのか

バイブコーディング (vibe coding) とは、自分でコードを一行ずつ書く代わりに、「こういうものが欲しい」と自然言語でAIに伝えて作ってもらうスタイルのことです。コードを書くスピードは劇的に上がりました。けれど、多くの人が同じ壁にぶつかります —— 「動かなくなったとき、自分では直せない」という壁です。この本は、その壁だけを壊すために書かれています。

AIは「速く書く」が、「正しいか」は保証しない

AIは驚くほど速く、それらしいコードを出します。しかしAIはあなたの意図を完全には知りません。存在しない関数を呼んだり、古いやり方を使ったり、一見動くのに肝心な部分だけ間違っていたりします。これは欠陥ではなく、現在のAIの性質です。だからこそ、出てきたものを受け取る側に判断力が必要になります。

プロのエンジニアとバイブコーダーの差は、もはや「コードを書けるか」ではありません。壊れたときに立て直せるかです。コードを生み出す力はAIが肩代わりしてくれた。だが「直す力」は、まだあなたの中にしか宿りません。そしてそれは、思っているよりずっと早く身につきます。

この本のたった一つの主張

バイブコーディングで挫折する最大の原因は「作れないこと」ではなく「直せないこと」。ゆえに、最短で効率よく学ぶべきは新しい文法ではなく、エラーを読み、原因を切り分け、検証する技術である。

「直せる人」が実際にやっている4つのこと

直せる人は天才なのではありません。たった4つの習慣を、無意識にこなしているだけです。

習慣	直せない人	直せる人
エラーが出たとき	パニックになり全文をAIに貼って「直して」	まずメッセージを読み、どこで何が起きたか特定する
原因の探し方	あちこち勘で書き換える	変更を1つずつ試し、原因を半分に絞り込む
AIへの頼み方	「動かない」とだけ言う	エラー全文・期待・実際・再現手順を渡す
動いた後	たぶん大丈夫、で次へ	本当に直ったか、別の所を壊していないか確認する

この4つを、第3章から第7章でひとつずつ、手を動かせる形に分解していきます。

この本の歩き方 —— 最短ルート

全部を一度に読む必要はありません。今あなたが「困っている」なら、まず次の3章だけ読めば、今日のエラーは越えられます。

- 1 今すぐ困っている人 → 第3章（エラーを読む）→ 第4章（切り分ける）→ 第10章のテンプレで質問する。
- 2 これから始める人 → 序章 → 第1章（心構え）→ 第2章（Gitで安全網）を先に。事故っても戻れる体を作る。
- 3 一通りできるが効率を上げたい人 → 第5章（指示の出し方）→ 第8章（罣カタログ）→ 第9章（ワークフロー）。

読み方のコツ

本書は「読む本」ではなく「使う本」です。各章の終わりにあるチェックリストと、第10章のテンプレートを、実際の作業中に開いて、手を動かしながら参照してください。1回通読して終わりにするより、10回引くほうが力になります。

前提知識はどこまで必要か

プログラミング経験はゼロでも読めるように書いています。ただし、次の3つだけは「言葉として知っている」状態にしておくとう理解が速くなります。すべて付録の用語集で1行解説しています。

- **ターミナル** 文字でコンピュータに命令を打つ黒い画面。AIが出すコマンドを実行する場所。

- **ファイルと拡張子** `.py` `.js` `.html` など、種類を表す末尾の文字。
- **エラー / 例外** プログラムが「これ以上進めない」と止まること。敵ではなく、案内役。

準備はこれだけです。次の章から、「直せる人の頭の中」を、ひとつずつあなたの中に移植していきます。

1 マインドセット —— あなたは「監督」である

道具を使いこなせるかどうかは、技術より先に立ち位置で決まります。 バイブコーディングでAIを「魔法使い」だと思っている限り、エラーは理不尽な災害に見えます。 AIを「速いが詰め甘い部下」だと捉えた瞬間、エラーは部下の報告書に変わり、あなたは落ち着いて指示を出せるようになります。

役割分担を正しく持つ

バイブコーディングは、あなたとAIの共同作業です。けれど対等ではありません。 **最終的な責任は、つねにあなたにあります。** なぜなら、その成果物を使うのも、壊れて困るのもあなただからです。

役割	AI（部下）の仕事	あなた（監督）の仕事
方向づけ	—	何を作るか、何を作らないかを決める
実装	コードを速く書く	意図どおりか判断する
検証	頼めば手伝う	本当に動くか最終確認する
責任	負えない	すべて引き受ける

合言葉

「AIに書かせる」のであって「AIに丸投げする」のではない。
監督は現場に立つ。出てきたものを観て、判断し、必要なら撮り直しを命じる。

3つの誤解を捨てる

誤解1：「AIが言うなら正しいはず」

AIは自信たっぷりに間違えます。これを **ハルシネーション**（もっともらしい嘘）と呼びます。 存在しない関数、動かない手順、古い書き方 —— どれも堂々と提示されます。「AIの出力は下書きであって、正解ではない」を初期設定にしてください。疑うのは失礼ではなく、仕事です。

誤解2：「エラー＝自分の失敗」

エラーは失敗ではなく **情報** です。プログラムが「ここで、これが理由で、進めません」と親切に教えてくれている状態です。エラーが出ないのに動きがおかしい方が、ずっと厄介です。エラーが出たら「手がかりをくれてありがとう」くらいの気持ちで読みましょう。

誤解3：「直すには全部理解しないといけない」

コード全体を理解する必要はありません。必要なのは「壊れている1箇所」までたどり着く力だけです。家の電気が消えたとき、家中の配線を理解しなくても、ブレーカーを見れば直せるのと同じです。第4章でその「ブレーカーの見つけ方」を学びます。

「怖い」を「手順」に変える

初学者がフリーズするのは、エラーが **感情の問題** になっているからです。赤い文字、英語、長い行——これらは「自分には無理」という感情を呼び起こします。解決策は、感情を手順に置き換えることです。怖くなったら、何も考えず次の順番を上から実行してください。

- 1 深呼吸して、コードに触らない。 いちばんやってはいけないのが、パニックで適当に書き換えること。
- 2 エラーメッセージの一番下の行を読む。多くの言語で、本当の原因はそこに書いてある（第3章）。
- 3 「どこで」起きたかを特定する。ファイル名と行番号が必ず添えられている。
- 4 直前に何を変えたか思い出す。9割の不具合は「最後に触った場所」が犯人。
- 5 それでも分からなければ、第10章のテンプレでAIに渡す。ただし丸投げではなく、情報を揃えて。

この章の核

立ち位置は「監督」。AIの出力は「下書き」。エラーは「情報」。直すのに「全理解」は要らない。

この4つを腹に落とすだけで、同じエラーがまったく違って見え始める。

章末チェック

- ☐ AIの出力を、正解ではなく「下書き」として見る準備ができた
- ☐ エラーが出ても、まずコードに触らず「読む」と決めた

- ☐ 「最後に変えた場所が犯人」という第一容疑者を覚えた

2 安全網を最初に張る —— Gitと小さなコミット

直せる人がいちばん最初にやるのは、コードを書くことではありません。「いつでも、壊れる前に戻れる」状態を作ることです。これがあると、思い切って試せます。失敗が怖くなくなります。バイブコーディングにおいて、Gitは技術ではなく**勇気の源**です。

なぜGitが「自己修正」の土台なのか

バイブコーディングでは、AIが一度に大量のコードを書き換えます。うまくいけば最高ですが、失敗すると「さっきまで動いていたのに、何が変わったのか分からない」状態に陥ります。Gitがあれば、この問いに一瞬で答えられます —— 「直前から、ここが、こう変わった」。そして、嫌なら一行のコマンドで巻き戻せます。

Gitなしで起きる悲劇

動いていた状態を保存せずにAIに大改造させ、壊れ、元に戻せず、何時間も溶かす。バイブコーディング最大の事故はこれです。逆に言えば、Gitさえあれば、この事故は原理的に起こりません。

覚えるのはこの5コマンドだけ

Gitには何百もの機能がありますが、自己修正の安全網として必要なのは、最初は次の5つだけです。

```
# ① 最初の一度だけ：このフォルダをGit管理下にする
git init

# ② 今の状態を「保存候補」に入れる（全ファイル）
git add -A

# ③ 保存する（セーブポイントを作る）。-m の後ろは自分用のメモ
git commit -m "ログイン画面が動くところまで"

# ④ 今、前回の保存から何が変わったかを見る
git diff

# ⑤ 【最強の戻る】保存以降の変更を全部捨て、直近の保存状態に戻す
git restore .          # 追跡中ファイルの変更を破棄
git checkout -- .      # 古いGitでの同等コマンド
```

コミット=ゲームのセーブポイント

`git commit` は、RPGのセーブと同じです。ボス（難しい機能）に挑む前にセーブする。全滅（コード崩壊）したら、ロード（戻す）してやり直す。「動いた」と思った瞬間が、セーブのタイミングです。

黄金ルール：「動いたら、すぐコミット」

バイブコーディングでの鉄則はたった一つ。小さく動くものができるたびに、すぐコミットすること。1日の終わりにまとめてではなく、「ボタンが表示された」「保存ができた」レベルで、こまめに。こうすると、いつ壊れても「直前の動く状態」がすぐ隣にあります。

タイミング	やること
新しい機能に手をつける前	今の動く状態をコミット（避難先を確保）
AIに大きな変更を頼む前	コミット（巻き戻し地点を作る）
小さな機能が1つ動いた	すぐコミット（成果を確定）
壊れて、原因が分からなくなった	<code>git restore .</code> で直近コミットへ撤退

事故ったときの「撤退」手順

AIにあれこれ頼んだ結果、何が何だか分からなくなった。よくあります。落ち着いて次を実行してください。

- 1 まず差分を見る： `git diff` で「保存後に何が変わったか」を確認。原因が見えることも多い。
- 2 戻すと決めたら： `git restore .`（または `git checkout -- .`）で、直近コミットの状態に全部戻す。
- 3 動くことを確認： 戻った状態でプログラムを動かし、確かに直っているか見る。
- 4 仕切り直す： さっきは一度に頼みすぎた、と反省し、今度はもっと小さく刻んで頼む（第9章）。

この章の核

コードを書く前に、まず `git init` → `git commit` で安全網を張る。「動いたらコミット」を口癖にする。壊れたら `git restore .` で戻る。
これだけで、バイブコーディング最大の事故が消える。

章末チェック

- ☐ プロジェクトで `git init` 済み、最初のコミットを作った
- ☐ 「動いたらすぐコミット」を今日から実践すると決めた
- ☐ `git restore .` で直近の動く状態に戻れることを確認した
- ☐ 大きな変更をAIに頼む前は、必ず先にコミットする習慣にした

3 エラーメッセージを読む技術

これは本書で最も価値のある章です。なぜなら、エラーの9割は、メッセージ自身が答えを書いているからです。読めないのは、英語が読めないからではありません。どこを、どの順で読むかを知らないだけです。順番さえ覚えれば、長い赤文字は「事件現場の地図」に変わります。

エラーメッセージは「3つの情報」でできている

言語が違って、エラーはほぼ必ず次の3点を含みます。まずこの3つを拾うことだけ考えてください。

拾う情報	意味	探し方
① 種類 (Type)	何系の問題か (名前が無い／型が違う／構文ミス…)	多くは行頭の <code>ooError</code>
② 場所 (Where)	どのファイルの何行目か	<code>file.py</code> や <code>line 42</code> の表記
③ 説明 (Message)	具体的に何が起きたか (人間向けの一文)	Error名のすぐ後ろ／一番下の行

読む順番 (これだけ覚える)

① 一番下から読む。② エラーの種類と説明文を拾う。③ 自分のファイル名がある行を探す。上から順に全部読もうとすると挫折します。多くの言語で、結論は一番下にあります。

実例で読んでみる (Python)

```
Traceback (most recent call last):
  File "app.py", line 3, in <module>
    print(user_name)
NameError: name 'user_name' is not defined
```

一番下から読みます。

- 説明: `name 'user_name' is not defined` = 「`user_name` という名前が定義されていない」

- 種類: `NameError` = 名前（変数）まわりの問題
- 場所: `app.py` の `line 3`

つまり「`app.py`の3行目で、`user_name` という変数を使おうとしたが、そんな変数は作られていない」。原因はたいてい **タイプミス**（`username` と書くべきだった）か、**定義し忘れ**。これで対処は決まります。

よく出るエラーの「翻訳辞書」

初学者が会えるエラーの大半は、次の顔ぶれの繰り返しです。これはそのまま使える早見表です。

エラー表記（言語）	日本語の意味	まず疑うこと
<code>NameError</code> / <code>ReferenceError</code> (Py/ JS)	その名前が見つからない	スペルミス・定義忘れ・スコープ外
<code>SyntaxError</code>	文法が壊れている	括弧 <code>()</code> や <code>{}</code> の閉じ忘れ、 ロン・カンマ
<code>TypeError</code>	型が想定と違う	文字列と数値の混同、 <code>None</code> / <code>undefined</code> を操作
<code>IndentationError</code> (Py)	字下げがおかしい	スペースとタブの混在、ずれ
<code>ModuleNotFoundError</code> / <code>Cannot find module</code>	ライブラリが見つからない	インストール忘れ（第8章）、名前 間違い
<code>KeyError</code> / <code>undefined (is not ...)</code>	その鍵・属性が存在しない	データの形が想定と違う、綴り違い
<code>IndexError</code> / <code>out of range</code>	範囲外を取りに行った	リストの数え方（0始まり）、空リ スト
<code>Permission denied</code>	権限がない	実行権限、書き込み禁止の場所
<code>Connection refused</code> / <code>ECONNREFUSED</code>	接続を断られた	サーバが起動してない、ポート違い
<code>404</code> / <code>500</code> (Web)	404=見つからない／500= サーバ側で爆発	404=URL/パス、500=サーバのコー ド・ログを見る

スタックトレースの「自分の行」を探す

長いエラー（スタックトレース）は、プログラムが通った道のりを上から並べたものです。その多くはライブラリ内部の行で、あなたが直す場所ではありません。コツは一つ。

魔法の一手

長いトレースの中から、自分が書いた／AIが書いたファイル名（例： `app.py` , `main.js` ）が出てくる一番下の行を探す。そこが、あなたが手を入れるべき現場です。ライブラリの奥深く（ `site-packages/...` や `node_modules/...` ）は、たいてい通り道にすぎません。

エラーが「出ない」不具合の見つけ方

やっかいなのは、エラーは出ないのに結果がおかしいケースです。こういうときは、自分でエラーの代わりを作ります。最も原始的で、最も効く道具が途中経過の表示です。

```
# 値が想定どおりか、要所要所で「のぞき見」する
print("ここまで来た。userの中身は:", user)          # Python
console.log("calcの結果:", result);                  # JavaScript
```

「思っていた値」と「実際に表示された値」がズレた地点 —— そこと、その直前が犯人です。プロも日常的にこれをやります。恥ずかしいことはありません。第4章でこの考え方を「二分探索」として体系化します。

エラーをそのまま検索・質問する技術

意味が分かっても解けないときは、調べる・聞く。その渡し方で結果が大きく変わります。

- 検索するなら： エラーメッセージの固有の部分（自分のファイル名や具体値）を消し、一般的な部分だけをコピーして検索する。例： `TypeError: cannot read properties of undefined reading map`。
- AIに聞いたら： エラー全文を省略せずそのまま貼る。一番下だけでなく全体を。さらに「何をしようとして」「何が起きたか」を添える（テンプレは第10章）。

やりがちなNG

エラーの一部だけスクショして「これ直して」と渡す。AIは場所も文脈も分からず、当てずっぽうで返してくる。テキストで全文＋直前にやったこと。これだけで返答の精度が跳ね上がる。

この章の核

エラーは「種類・場所・説明」の3点セット。読むのは一番下から。長いトレースは自分のファイル名の最下行が現場。エラーが出ない不具合は`print/console.log`でのぞく。

章末チェック

- ☐ エラーを見たら、まず一番下の行から読む習慣がついた
- ☐ 「種類・場所・説明」の3点を拾えるようになった
- ☐ 早見表で、よく出るエラーの大意がつかめる
- ☐ 長いトレースから「自分のファイル名の行」を探せる
- ☐ エラーが出ない不具合に `print` / `console.log` を使える

4 体系的デバッグ —— 推測をやめ、検証する

直せない人は推測します。「たぶんここが原因かな」とあちこち書き換え、いつの間にか別のバグを増やします。直せる人は検証します。仮説を立て、一つずつ確かめ、原因を物理的に追い詰めます。デバッグはセンスではなく手順です。この章でその手順を丸ごと渡します。

デバッグの黄金ループ

あらゆるバグ取りは、結局この4ステップの繰り返しです。迷ったら、ここに戻ってください。

- 1 観察：何が起きているか、事実だけを書き出す。「期待＝〇〇、実際＝××」。
- 2 仮説：「〇〇が原因では？」と1つだけ立てる。複数同時に疑わない。
- 3 検証：その仮説だけを確かめる小さな実験をする（値を表示する／一行だけ変える）。
- 4 判定：当たりなら直す。外れなら仮説を捨て、観察に戻る。ここで元に戻すのが超重要。

最大の禁手「同時に2つ変える」

原因が分からないからと、一度に複数箇所をいじると、直ってもなぜ直ったか分からず、また同じ沼にはまります。一度に変えるのは1つだけ。外れたら戻す。これを守るだけで、デバッグ速度が倍増します。

武器1：二分探索（はさみうち）

「どこかで値がおかしくなる」「処理のどこかで止まる」—— 範囲が広いときの最強の手法です。全部を疑う代わりに、真ん中で半分に割る。これを繰り返すと、100行のコードでも7回で犯人に届きます。


```
処理A → 処理B → 処理C → 処理D → 処理E    # どこかで壊れている
```

```
# 真ん中 (C) の直後で値をのぞく
```

```
print("Cの後の値:", value)
```

```
→ ここで正常なら、犯人は D か E (後半)
```

```
→ ここで異常なら、犯人は A か B か C (前半)
```

```
# 生き残った半分の、さらに真ん中でまた割る。これを繰り返す。
```

コードだけでなく「変更」も二分探索できる

「昨日まで動いていたのに今日壊れた」なら、変更そのものを半分ずつ戻して原因を特定できます。Gitでこまめにコミットしていれば（第2章）、どのコミットで壊れたかを `git diff` で挟み撃ちできます。

武器2：最小再現（小さく切り出す）

大きなプログラムの中でバグを追うのは大変です。そこで、問題の部分だけを取り出した、ごく小さなコードを作ります。これを最小再現コードと呼びます。作る過程で原因が見えることも多く、AIに渡すときも精度が段違いになります。

```
# 元：100行のアプリの中で、日付の計算だけがおかしい
# → 日付計算の部分だけ、5行に切り出して単体で動かす
from datetime import date
d = date(2026, 6, 8)
print(d)                # まずここで正しいか確認
print(d.weekday())      # 次にここ。月曜=0 を知らないと「ズレてる！」と誤解する
```

武器3：仮定を疑う（思い込みリスト）

バグが見つからないとき、たいてい原因は「正しいと信じて確認していない場所」にあります。次のような当たり前すぎて疑っていないことを、声に出して確認します。

思い込み	実は…
「このファイルを実行している」	別のファイル／古いファイルを動かしていた
「保存した」	保存し忘れて、前のコードが動いていた
「この値はXのはず」	表示してみたら全然違う値だった
「変更は反映されている」	サーバ再起動が必要で、古い状態のままだった
「正しいコードを編集している」	コピーした別の場所を直していた

魔法の質問

詰まったら自分にこう聞く —— 「本当に？それ、自分の目で確認した？」。 “確認したつもり” を一つずつ “実際に確認” に変えていくと、たいてい犯人が顔を出します。

AIにデバッグを手伝わせる正しい頼み方

AIは優秀なデバッグ相棒ですが、**丸投げ**すると暴走します。主導権はあなたが握ったまま、こう使います。

- ・「直して」ではなく「原因の候補を3つ挙げて」と頼む。判断はあなたがする。
- ・「このコードは何をしているか説明して」で、まず理解を補強する。
- ・「ここに print を入れるなら、何を表示すべき？」と、検証の設計を手伝わせる。
- ・AIが大改造を提案してきたら**一旦止める**。原因が分かる前の大改造は、傷口を広げる。

この章の核

推測ではなく検証。一度に変えるのは1つ、外れたら戻す。 広い範囲は二分探索で半分に割る。複雑な問題は最小再現で小さくする。そして「確認したつもり」を疑う。これがデバッグの全技術。

章末チェック

- ☐ 観察→仮説→検証→判定のループを意識できる
- ☐ 「一度に1つだけ変える」を徹底できる
- ☐ 二分探索で原因の範囲を半分に絞れる

- ☐ 問題部分だけの最小再現コードを作れる
- ☐ 詰まったら「本当に？確認した？」と自問できる

5 AIに正しく指示する —— コンテキスト設計

AIの出力の質は、ほぼ完全に入力の質で決まります。同じAIでも、頼み方ひとつで、使い物にならないコードを返すか、一発で意図どおりのコードを返すかが変わります。「直す回数を減らす」最良の方法は、そもそも **正しく頼んで、壊れにくくすること**です。

良い指示の「5要素」

うまくいく依頼には、ほぼ必ず次の5つが入っています。困ったら、この型に流し込んでください。

要素	中身	悪い例 → 良い例
① 目的	最終的に何がしたいか	「フォーム作って」→「問い合わせを受け付けるフォームが欲しい」
② 文脈	使っている言語・道具・既存コード	(無し) → 「Python・Flask・既存の <code>app.py</code> に追加」
③ 制約	守ってほしい条件・禁止事項	(無し) → 「新しいライブラリは増やさず標準機能で」
④ 入出力	入力例と、期待する出力例	(無し) → 「名前とメールを受け取り、確認画面を返す」
⑤ 形式	返してほしい形	(無し) → 「全文ではなく変更箇所だけ。理由も一言」

ビフォー → アフター

×：「ログインを作って。」

○：「Python+Flaskで、メールとパスワードのログイン機能を、既存の `app.py` に追加したい。パスワードは平文保存せずハッシュ化したい（標準の `werkzeug` で）。 入力＝メールとパスワード、成功＝トップヘリダイレクト、失敗＝『認証に失敗しました』を表示。 変更したファイルと、追記する行だけ見せて。」

大きく頼まない —— 「1回1機能」

最もよくある失敗は、一度に欲張ること。「ログインも登録もマイページも全部」と頼むと、AIは巨大なコードを返し、どこかが壊れ、どこが原因か追えなくなります。小さく頼めば、出力も小さく、確認も巻き戻しも簡単です。

分割の目安

「動作確認が1回でできる大きさ」まで割る。ログイン機能なら、①入力フォームを表示 → ②送信を受け取る → ③照合する → ④結果を出す、と4回に分ける。一段ごとに動かして確認し、コミットしてから次へ。

AIの「記憶」を管理する

長い会話を続けると、AIは前提を忘れたり、混乱したりします。これは正常な挙動です。対処法を知っておきましょう。

- ・話が噛み合わなくなったら、現状を再提示する。「今のコードはこれ（貼る）。この状態から、次は〇〇したい」。
- ・別の話題に移るときは、新しい会話を始める。古い文脈を引きずらせない。
- ・重要な決定（使う言語・方針）は、毎回冒頭で明示する。「Python・ライブラリ追加なし、を厳守」。
- ・AIが古い情報を前提にしていたら、訂正して固定する。「いま使っているのは v2 です。v1の書き方はしないで」。

「説明させて、理解してから受け取る」

コードを受け取る前に、ひと手間。「このコードが何をしているか、3行で説明して」と頼みます。これには2つの効果があります。①あなたの理解が進む（直すときに効く）。②AI自身が説明する過程で、矛盾に気づき品質が上がる。

理解せずに貼らない

意味の分からないコマンドを、AIに言われるまま実行するのは危険です。とくに `rm`（削除）、`sudo`（管理者権限）、外部ヘータを送るコードは、必ず何をするか説明させてから。「これは何をするコマンド？危険はない？」の一言が、取り返しのつかない事故を防ぎます。

修正を頼むときの黄金フォーマット

不具合の修正依頼は、次の4点を揃えるだけで、返答の精度が劇的に上がります（第10章にテンプレあり）。

- 1 やりたかったこと：「保存ボタンでデータを保存したい」
- 2 実際に起きたこと：「押すと画面が真っ白。エラー全文は↓（貼る）」
- 3 関係するコード：該当ファイルの中身（省略せず）
- 4 試したこと：「〇〇は試したが変化なし」（同じ提案の再来を防ぐ）

この章の核

出力の質＝入力の質。目的・文脈・制約・入出力・形式の5要素で頼む。1回1機能に割る。受け取る前に説明させる。修正依頼は意図・実際・コード・試行の4点セット。

章末チェック

- ☐ 依頼に「目的・文脈・制約・入出力・形式」を入れられる
- ☐ 機能を「動作確認1回分」まで小さく割れる
- ☐ 受け取る前にコードの説明をAIにさせている
- ☐ 危険なコマンドは実行前に必ず確認している
- ☐ 修正依頼を4点セットで出せる

コードを「読める」最低限

直すために、コードを **書ける** 必要はありません。必要なのは、AIが出したコードを見て「**だいたい何が起きているか**」を追えること。全文の翻訳ではなく、流れを追う「速読」です。ここでは、言語が違って共通する“**読みの骨格**”だけを、最短で渡します。

プログラムは「上から下へ」流れる

ほとんどのコードは、**上から順に、一行ずつ**実行されます。まずこの感覚を持つだけで、「どこまで進んで、どこで止まったか」が追えます。エラーの行番号が効くのも、この性質のおかげです。

どの言語にもある「5つの部品」

言語が違って、コードは結局この5部品の組み合わせです。これさえ見分けられれば、流れは追えます。

部品	役割	見た目（例）
変数	値に名前をつけて入れる箱	<code>name = "佐藤"</code> / <code>let count = 0</code>
関数	処理のまとまり。呼ぶと動く	<code>def save(x):</code> / <code>function save(x){}</code>
条件分岐	「もし～なら」で道を分ける	<code>if age >= 18:</code>
繰り返し	同じ処理を何度も回す	<code>for item in list:</code>
呼び出し	他の機能を使う	<code>save(user)</code> / <code>print(x)</code>

5部品で読んでみる

```
total = 0                                # 変数：合計を入れる箱、最初は0
for price in cart:                       # 繰り返し：カートの中の商品について
    total = total + price                # 合計に足していく
if total >= 5000:                        # 条件：もし合計が5000以上なら
    print("送料無料")                  # 送料無料と表示
```

日本語にすると「カートの値段を全部足して、5000以上なら送料無料と出す」。一語一語の文法を知らなくても、**流れ**は読めます。これで十分です。

「名前」を信じて読む

良いコードは、変数や関数の**名前**が中身を語ります。`send_email` はメールを送る、`is_valid` は正しいかどうかを表す。細部が分からなくても、名前から役割を推測して流れを追えます。逆に、AIが付けた名前と動きが食い違っていたら、それ自体がバグのサインです。

分からない一行は「AIに通訳させる」

読めない行が出てきたら、止まらずにこう聞きます —— 「この行が何をしているか、初心者向けに1文で説明して」。これを繰り返すうちに、同じパターンが読めるようになり、聞く回数は自然に減ってきます。

読みの3点セット質問

- ① 「このコード全体を、3行で要約して」（地図をもらう）
- ② 「`oo` の行は何のため？」（気になる所だけ深掘り）
- ③ 「もしこの行を消したら何が起きる？」（その行の存在理由を知る）

「データの形」を意識する

不具合の多くは、**データの形の思い違い**から生まれます。一個の値なのか、リスト（並び）なのか、辞書（鍵と値の対）なのか。これを取り違えると `TypeError` 系が出ます。読むときは「今この変数には、どんな形のデータが入っている？」を常に意識します。迷ったら表示して確かめる（第3章）。


```
user = "佐藤"                # 1個の文字列
users = ["佐藤", "鈴木"]    # リスト（並び）。users[0] が "佐藤"
user = {"name": "佐藤", "age": 30} # 辞書。user["name"] が "佐藤"
```

この章の核

コードは上から下へ流れる。中身は5部品（変数・関数・条件・繰り返し・呼び出し）。名前から役割を推測し、分からない行はAIに通訳させる。データの形を常に意識する。書けなくても、これで「追える」。

章末チェック

- ☐ コードが上から順に実行されると理解した
- ☐ 5部品（変数・関数・条件・繰り返し・呼び出し）を見分けられる
- ☐ 名前から関数や変数の役割を推測できる
- ☐ 分からない行をAIに1文で説明させられる
- ☐ 変数に入っているデータの「形」を意識できる

7 検証とテスト —— 「動いた」を証明する

バンプコーディング最大の落とし穴は、「動いたつもり」です。AIが「できました」と言い、画面が一度それらしく動いた —— それだけで完成にすると、後で必ず痛い目を見ます。直せる人は「動いた」を証明します。この章は、その証明のやり方を、軽いものから順に渡します。

検証の3階層 —— まずは一番下でいい

階層	やり方	いつ使う
① 目視確認	自分で操作して、目で見て確かめる	常に。最低限これは必ず
② 境界の確認	「変な入力」でも壊れないか試す	機能が一通り動いたら
③ 自動テスト	確認をコード化し、毎回自動で回す	同じ確認を繰り返すなら

最初は①だけで十分です。慣れてきたら②、繰り返しが増えてきたら③へ。背伸びは不要です。

① 目視確認 —— ただし「正常系」で満足しない

ほとんどの人は、うまくいくケース（正常系）だけ試して安心します。しかしバグは、たいていうまくいかないケースに潜んでいます。最低限、次の3パターンは触ってください。

- ・ 正常：普通に正しく使う（例：正しいメールで登録）
- ・ 空：何も入れずに実行（例：空欄のまま送信）
- ・ 変：想定外を入れる（例：メール欄に「あ」、数量にマイナス）

「壊し方」チェックリスト（汎用）

- 入力を空のまま実行したら？
- とても長い文字や絵文字を入れたら？
- 数字の欄に文字、マイナス、0、巨大な数を入れたら？
- 同じボタンを連打したら？ 二重に登録されない？
- 途中で戻る・リロードしたら？ データは壊れない？

② 境界値 —— バグは「端っこ」に住む

不具合が集まる場所は決まっています。**境目**です。「18歳以上OK」なら、17・18・19歳。「10件まで」なら、9・10・11件目。 ちょうどの値とその前後を試すと、`>` と `>=` の取り違えのような典型バグが面白いほど見つかります。

③ 自動テスト —— 「確認」を未来の自分に贈る

同じ手動確認を何度も繰り返すなら、それをコードにしておくと、毎回ボタン一つで確認できます。これが**自動テスト**です。難しく考えず、「この入力なら、この結果になるはず」を書くだけです。

```
# 例：税込価格を計算する関数 calc_tax をテストする (Python)
def test_calc_tax():
    assert calc_tax(100) == 110      # 100円 → 110円のはず
    assert calc_tax(0) == 0          # 0円 → 0円のはず（端っこ）
    assert calc_tax(1) == 1          # 端数の丸めも確認
```

`assert A == B` は「AがBでなければ、そこで止まって教えて」という意味。 これを書いておけば、後でAIにコードを改造させた後でも、**壊れていないか一瞬で分かります**。 バンプコーディングと自動テストは、実はとても相性が良いのです。

AIにテストを書かせる

テストの書き方が分からなければ、こう頼みます —— 「この関数のテストを書いて。正常系・空・異常系・境界値を含めて」。 さらに「**テストを先に書いて、それが通るように実装して**」と頼むと、品質がぐっと安定します。

「直った」を確認する —— 回帰チェック

バグを直したときは、2つを必ず確認します。①本当にそのバグが消えたか。そして見落としがちな②別の場所を壊していないか。AIの修正は、しばしば**関係ない所に副作用**を出します。直したら、周辺の機能も一通り触り直す —— これを**回帰チェック**と呼びます。自動テストがあれば、これが一瞬で済みます。

「たぶん動く」を完成にしない

動作を一度も自分で確認せずに「完成」「デプロイ」するのは、最も多い後悔の原因です。AIの「できました」は報告であって、保証ではありません。最後に動かすのは、いつもあなた。

この章の核

「動いたつもり」を証明に変える。最低限正常・空・変の3パターンを試す。バグは境界に住む。繰り返す確認は自動テスト化する。直したら回帰チェックで副作用を見る。

章末チェック

- ☐ 正常系だけでなく、空・異常・境界値も試している
- ☐ 「壊し方チェックリスト」で意地悪な入力を試せる
- ☐ 簡単な `assert` テストを書ける（またはAIに書かせられる）
- ☐ バグ修正後に「別の場所を壊していないか」確認している
- ☐ 自分で動作確認するまで「完成」と言わないと決めた

8

よくある罣カタログ

バンプコーディングでハマる落とし穴は、実はパターンが決まっています。ここに代表的な罣を集めました。「これ、今の自分だ」と思ったら、その場で対処法に飛んでください。事前に知っているだけで、ハマる時間が数時間から数分に縮みます。

罣1：ハルシネーション（存在しないものを使う）

AIが、実在しない関数・ライブラリ・APIを、堂々と使ってくることがあります。

- ・サイン： `ModuleNotFoundError`、`AttributeError`、「そんな関数ない」系のエラー。
- ・対処： その名前を公式ドキュメントで検索。無ければAIに「それは実在する？公式の名前を確認して」と問い直す。実在する代替を出させる。

罣2：依存関係地獄（ライブラリが入っていない／合わない）

「動くはずのコード」が `ModuleNotFoundError` で動かない。多くはインストール忘れです。

```
# 「〇〇が見つからない」と言われたら、まずインストール
pip install 〇〇          # Python
npm install 〇〇          # JavaScript / Node.js
```

バージョンずれの対処

「昔の書き方」をAIが使い、新しいライブラリで動かないことがあります。エラーに `deprecated`（非推奨）や「〇〇 has no attribute」が出たら、AIに「使っているのは 〇〇 の v△ です。その版での正しい書き方にして」と版を伝える。

罣3：スコープ・クリープ（いつの間にか巨大化）

小さく始めたのに、AIに頼むたび機能が増え、コードが膨らみ、どこで何が起きているか分からなくなる現象。 対処：「動作確認1回分」を超えそうになったら立ち止まる（第5・9章）。今いない機能は「後で」と断る勇気を持つ。

罣4：サイレント上書き（黙って別の所が変わる）

AIに「Aを直して」と頼んだら、ついでにBやCも書き換えられていた、というケース。 対処：変更を受け取ったら必ず `git diff` で全変更点を見る（第2章）。 意図しない変更があれば戻す。「ここだけ変えて、他は触らないで」と指示に明記する。

罣5：セキュリティの穴

動けばOK、で見落とされがちですが、これは取り返しがつかない事故につながります。最低限、次の3つだけは守ってください。

やってはいけない	正しくは
APIキー・パスワードをコードに直書き	環境変数や設定ファイルに分け、Gitに上げない（ <code>.gitignore</code> ）
秘密情報を含むファイルをそのまま公開リポジトリへ	公開前に「鍵やパスワードが入っていないか」を必ず確認
ユーザー入力を無検証でDB・コマンドに渡す	「この入力、危険な使われ方をしない？」とAIに監査させる

公開前の3秒チェック

GitHubなどに上げる前に必ず —— 「パスワード・APIキー・個人情報が含まれていないか?」。 一度公開した秘密は、消しても他人にコピーされ得ます。AIに「このコードに公開してはいけない情報は?」と聞くのも有効。

罣6：同じ堂々巡り（AIと無限ループ）

AIに直してもらおう→別の所が壊れる→直す→また壊れる…の無限ループ。これはAIが全体像を見失っているサインです。

- 1 一旦止める。これ以上頼んでも泥沼。
- 2 動く所まで戻す。 `git restore .` で直近の正常状態へ（第2章）。
- 3 新しい会話で、現状を一から提示し直す。混乱した文脈を捨てる（第5章）。
- 4 もっと小さく刻んで頼み直す。一度に1つだけ。

罣7：「コピペで動いた」を理解した気になる

動いたコードを、中身を理解せずに積み上げると、壊れたときまったく手が出せなくなります。対処：新しいコードを受け取るたび、第6章の「3点セット質問」で要点だけでも掴んでおく。完全理解は不要だが、「ここが何の係か」は知っておく。後の自分を助けます。

この章の核

ハマりはパターン。実在を疑い（罣1）、入れ忘れを疑い（罣2）、肥大化を断り（罣3）、`git diff` で監視し（罣4）、秘密を守り（罣5）、無限ループは戻ってやり直す（罣6）。知っているだけで、沼の時間が桁で縮む。

章末チェック

- ☐ 実在しない関数・ライブラリを疑い、確認できる
- ☐ `ModuleNotFoundError` はインストールから疑える
- ☐ 変更後は `git diff` で意図しない変更を見ている
- ☐ APIキーやパスワードをコードに直書きしていない
- ☐ AIと堂々巡りになったら、戻って仕切り直せる

9 実践ワークフロー —— 1機能を完成させる流れ

ここまでの技術を、ひとつの流れに組み立てます。これは、直せる人が 新しい機能を1つ作る
ときに、無意識でなぞっている手順です。そのまま真似してください。最初は遅く感じて
も、これが結局いちばん速い道です。

1機能を作る9ステップ

- 1 ゴールを一文で書く。「ユーザーが問い合わせを送れるようにする」。曖昧なら、まだ始めない。
- 2 動作確認1回分まで割る。「①フォーム表示 → ②送信受取 → ③保存 → ④完了表示」。今日は①だけでもいい。
- 3 始める前にコミット。`git add -A && git commit -m "〇〇の前"`。避難先を確保（第2章）。
- 4 5要素でAIに頼む。目的・文脈・制約・入出力・形式（第5章）。小さく、1ステップ分だけ。
- 5 受け取る前に説明させる。「これは何をやる?危険はない?」（第5・6章）。
- 6 動かして確認する。正常・空・変の3パターン（第7章）。エラーが出たら第3・4章へ。
- 7 動いたら即コミット。セーブポイント確定。これで①が「壊れない資産」になる。
- 8 次のステップへ。②に進み、4〜7を繰り返す。一段ずつ確実に。
- 9 全部つながったら、通しで回帰チェック。端から端まで動かし、副作用を確認（第7章）。

このループの肝

「頼む → 理解する → 確かめる → 保存する」の小さな円を、何度も回すこと。一周がどんどん小さく速くなるほど、事故っても失うものが減り、原因も一瞬で分かるようになります。バイブコーディングが速い人は、大きく作る人ではなく、**小さい円を高速で回す人**です。

つまづいたときの「判断フロー」

作業中に手が止まったら、考え込まず、この分岐を上から当てはめてください。

状況	すぐやること	参照
エラーが出た	一番下から読む → 種類・場所・説明を拾う	第3章
エラーは無いが結果が変	<code>print</code> / <code>console.log</code> で値をのぞく	第3・4章
原因の範囲が広い	二分探索で半分に割る	第4章
AIと堂々巡り	<code>git restore .</code> で戻り、小さく頼み直す	第2・8章
何を直せばいいかわからない	最小再現を作る → 4点セットでAIに渡す	第4・10章
動いたが不安	境界値＋回帰チェック	第7章

「行き詰まりタイマー」を使う

一人で（AIと）粘る時間には上限を決めます。おすすめは15分ルール。 同じ問題で15分進展がなければ、やり方を変えます —— 戻る、最小化する、別の聞き方をする、休む。 同じ方法で粘り続けるのが、最も時間を溶かします。

休憩は最強のデバッグ

どうしても解けないとき、5分席を立つと、戻った瞬間に原因が見えることがよくあります。 脳が裏で整理してくれるからです。詰まったら粘らず、一度離れる —— これも立派な技術です。

1日の終わりにやること

- ・ 動く状態でコミットして終える。 翌日「動く所」から再開できる。
- ・ 次にやることを一行メモ。 「次：保存機能の異常系を試す」。再開が一瞬になる。
- ・ 未解決の謎を書き残す。 寝かせると翌朝あっさり解けることがある。

この章の核

頼む→理解→確かめる→保存の小さな円を高速で回す。 詰まったら判断フローを上から当てる。15分ルールで粘りすぎない。 1日を動く状態で終える。これがバイブコーディングの基本動作。

章末チェック

- ☐ 機能を「動作確認1回分」のステップに割れる

- ☐ 各ステップで「頼む→理解→確かめる→保存」を回せる
- ☐ 詰まったら判断フローで次の一手を選べる
- ☐ 15分ルールで、粘りすぎを止められる
- ☐ 毎日を「動く状態＋次の一行メモ」で終わっている

テンプレート & チェックリスト集

ここは「読む」ページではなく「使う」ページです。作業中にこの章を開き、枠をそのままコピーして、[] を自分の状況で埋めてください。良い質問ができれば、AIは別人のように良い答えを返します。質問の質を、テンプレートで底上げします。

テンプレ① エラーを直してもら（最重要）

コピーして使う

- やりたかったこと
[例：保存ボタンを押したらデータが保存される]
- 実際に起きたこと
[例：押すと画面が真っ白になる]
- エラーメッセージ（全文・省略しない）
[ここにエラーを丸ごと貼る]
- 関係するコード（省略しない）
[該当ファイルの中身を貼る]
- 使っている環境
[例：Python 3.12 / Flask / Mac]
- すでに試したこと
[例：再起動した、○○を変えた → 変化なし]
- お願い
原因の候補をいくつか挙げ、いちばん可能性が高いものから、
変更すべき箇所だけを、理由とともに教えてください。
いきなり全体を作り直さないでください。

テンプレ② 新しい機能を頼む

コピーして使う

■ 目的（最終的にしたいこと）

[例：問い合わせフォームを作りたい]

■ 環境・既存コード

[例：Python + Flask。既存の `app.py` に追加したい（↓に貼る）]

■ 制約（守ってほしいこと）

[例：新しいライブラリは増やさない／既存の他の機能は触らない]

■ 入力と期待する出力

入力：[例：名前・メール・本文]

成功時：[例：「送信しました」と表示]

失敗時：[例：「未入力の項目があります」と表示]

■ お願い

まず全体の方針を3行で説明してから、

今回は「[例：フォームを表示する]」の部分だけを実装してください。

変更したファイルと追記する箇所だけ見せてください。

テンプレ③ コードを理解する

コピーして使う

このコードについて、初心者向けに教えてください。

- 1) 全体が何をするか、3行で要約
- 2) [気になる行] は何のためにあるか
- 3) 想定外の入力 cameたら、どこで壊れそうか
- 4) もし1つだけ改善するなら、どこか（理由も）

[ここにコードを貼る]

テンプレ④ 安全確認（実行前）

コピーして使う

次のコマンド／コードを実行しようとしています。

- 1) これは具体的に何をしますか
- 2) 取り返しのつかない操作（削除・上書き・外部送信）を含みますか
- 3) 含むなら、安全に試す方法がありますか

[ここにコマンド／コードを貼る]

万能チェックリスト集

● プロジェクト開始時

- ☐ `git init` して最初のコミットを作った
- ☐ 作るものの「ゴールを一文」で書いた
- ☐ 「動作確認1回分」のステップに割った
- ☐ APIキー等を入れるなら `.gitignore` を用意した

● AIにコードをもらった直後

- ☐ 何をするコードか、要点を理解した（テンプレ③）
- ☐ 危険な操作が含まれていないか確認した（テンプレ④）
- ☐ 意図しない箇所まで変わっていないか `git diff` で見た

● エラーが出たとき

- ☐ コードに触る前に、まず一番下から読んだ
- ☐ 種類・場所・説明の3点を拾った
- ☐ 直前に変えた場所を第一容疑者にした
- ☐ 解けなければ、テンプレ①で情報を揃えて聞いた

● 「完成」と言う前

- ☐ 正常・空・変の3パターンを自分で動かした

- ☐ 境界値（端っこ）を試した
- ☐ 直した後、周辺機能の回帰チェックをした
- ☐ 動く状態でコミットした
- ☐ 公開するなら、秘密情報が混ざっていないか確認した

この章の使い方

4つのテンプレを、すぐ呼び出せる場所（メモアプリ等）にも貼っておく。作業中はこの章を開きっぱなしにして、迷ったらチェックリストを上から潰す。質問の型と確認の型を、道具として常備するのが、最短で安定する近道。

付 付録 —— 用語集 / 30分クイックスタート

最後に、つまずきやすい言葉を「ひと言」で引ける用語集と、今日この場で安全網を張るための30分クイックスタートを置きます。細かい正確さより、とりあえず話が通じることを優先した説明です。

30分クイックスタート —— 今日やる最小セット

まだ何も準備していない人へ。この順でやれば、30分で「事故らない体」になります。

- 1 (10分) 安全網を張る。プロジェクトのフォルダで `git init` → `git add -A` → `git commit -m "最初の状態"`。これで、いつでも戻れる起点ができた（第2章）。
- 2 (5分) テンプレを保存する。第10章のテンプレ①（エラー修正）とテンプレ④（安全確認）を、メモアプリにコピーしておく。
- 3 (10分) 小さく1つ作る。ごく小さな機能を、第9章の9ステップで1周だけ回す。動いたらコミット。
- 4 (5分) わざと壊して、戻す。コードを適当に壊し、`git restore .` で戻せることを体感する。これで「壊すのが怖くない」状態が手に入る。

最初の一步で十分

全部を完璧にやろうとしないこと。「戻れる」と「小さく回す」の2つさえ体に入れば、あとは実戦の中で、この本の各章を必要なときに開けばいい。学びは、走りながらで間に合います。

用語集 —— ひと言でわかる

用語	ひと言の意味
バイブコーディング	自然言語でAIに伝えてコードを作ってもらう開発スタイル
ハルシネーション	AIがもっともらしく出す“嘘”。実在しない関数などを堂々と使う
ターミナル / シェル	文字で命令を打つ画面。AIが出すコマンドを実行する場所
コマンド	ターミナルに打つ命令文（例： <code>pip install</code> ）
エラー / 例外	プログラムが「進めない」と止まること。原因のヒント付き
スタックトレース	エラーに至るまでにプログラムが通った道のりの一覧
デバッグ	不具合（バグ）の原因を見つけて直す作業
二分探索	範囲を半分ずつに割って原因を絞り込むやり方
最小再現コード	問題部分だけを取り出した、ごく小さなコード
Git	変更履歴を記録し、いつでも過去に戻れるようにする道具
コミット	その時点の状態を保存すること。ゲームのセーブ
diff（差分）	前の状態から「どこが変わったか」を示したもの
リポジトリ	Gitで管理しているプロジェクト一式の入れ物
ライブラリ / パッケージ	誰かが作った便利な部品。入れて使う
依存関係（dependencies）	そのコードが動くのに必要な外部の部品たち
インストール	必要な部品を取り込んで使えるようにすること
API	外部の機能呼び出すための窓口・取り決め
APIキー	その窓口を使うための秘密の鍵。絶対に公開しない
環境変数	鍵などの秘密を、コード外に分けて持たせる仕組み
変数	値に名前をつけて入れておく箱
関数	処理のまとめ。呼ぶと中身が動く

引数 / 戻り値	関数に渡す材料 / 関数が返してくる結果
型 (type)	データの種類 (文字列・数値・リストなど)
None / null / undefined	「値が無い」状態。これを操作するとよくエラーになる
リスト / 配列	値を順番に並べた入れ物。0番から数える
辞書 / オブジェクト	「鍵」と「値」の対でデータを持つ入れ物
境界値	条件の“ちょうど境目”の値。バグが集まりやすい
回帰チェック	修正後に「別の所を壊していないか」確認すること
正常系 / 異常系	うまくいく場合 / うまくいかない場合の挙動
テスト	「この入力ならこの結果」を確かめる仕組み
deprecated (非推奨)	「古いやり方なのでそのうち使えなくなる」という警告
デプロイ	作ったものを、実際に使える場所へ公開すること
.gitignore	Gitに含めたくないファイルを指定する設定

おわりに —— 「直せる」はあなたの自由になる

この本が一貫して伝えてきたのは、たった一つのことです。 バイブコーディングの主役は、コードを生み出すAIではなく、それを見て、確かめて、直せるあなただということ。

作る速さは、もう手に入りました。次に手に入れるべきは、壊れても怖くないという自由です。 その自由は、新しい才能ではなく、この本に書かれた地味な手順の習慣化から生まれます。 エラーを一番下から読む。一度に1つだけ変える。動いたらコミットする。確認するまで完成と言わない。

どれも派手ではありません。けれど、これを淡々と続けられる人だけが、 AIを「魔法」ではなく「道具」として使いこなし、自分の作りたいものを、最後まで自分の手で完成させられます。

最後に、これだけ持ち帰る

戻れるようにして (Git)、小さく回し (ワークフロー)、エラーは読み (第3章)、推測せず検証する (第4章)。

この4つができれば、あなたはもう「自分で直せる人」です。あとは、手を動かすだけ。

—— バイブコーディング自己修正の教科書 ・ 了 ——